

COMP101

Introduction to Programming

2025-26

Lecture-08

Formatting of Output

Slides 1 to 10 show the current formatting style you should use

Slides 11 to 16 are for computational thinking

Slides 17 to the end are for information only – they explain older formatting styles

Do not use these old styles.

Formatting Output

- When we print() to screen, output format is a pre-defined standard
- The output standard depends on type (numeric or string)
- Numeric type standard
 - output truncated or untruncated depending on type and operation used
 - e.g. we may not need to display to 16dp when 2dp or 3dp is sufficient
 - Integers output left justified in field length enough to show the entire integer
 - Floats output left justified in field length 18 places
 - Gives 16dp after the decimal point
- String type standard
 - output left justified in a field length big enough to show all the string

Formatting styles

- ‘Formatting styles’ allow us to control output
 - We can over-ride standard output to output data our own way
- Python has three different formatting styles depending on Python version
 - f-strings (Python v.3.6 upwards) **Use this one for COMP101**
 - Latest style but incompatible with versions before v3.6
 - str.format (Python v.3.x up to v.3.5.x) **Don't use for COMP101**
 - Confusing and hard to use
 - % format (Python v.2.x) **Don't use it – at all**
 - Out of date and incompatible with higher versions

f-string ('Formatted String Literals')

- As you read and search around, authors may use jargon differently.

- Our definitions are as follows:

- String Literal

```
print ("Hello World")
```

Something that has been entered as part of the code and not typed in by the user in response to an input() prompt

- Formatted string literal or 'f-string'

Is a string literal that has output formatting fields in it

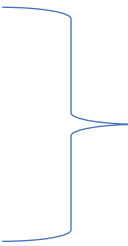
This is where Python gets the name for the style 'f-string'

f-string

- f-string format makes use of two types of ‘brackets’ from the list below – we need to get them in the right place for the formatting to work

() Parentheses ‘round brackets’

{ } Braces ‘curly brackets’



We use these two brackets in f-string formatting

[] Box ‘square brackets’

< > Angle don’t confuse with less than/greater than

<< >> Guillemets for syntactic analysis

f-string

- Evaluated at run-time – gives faster, more concise and readable code
 - Expressions within the braces are evaluated to their own scope then put with the string literal part of the f-string
 - No need to specify 's' for string or 'd' for integer or 'f' for float (older styles)
- f-string is invoked with f or F at start of print() function

```
print (f"Some text {var}")
```

OR

```
print (F"Some text {var}")
```

f-string – formatting strings

```
a = input("Enter a word: ") one  
b = input("Enter another word: ") two
```

```
print(f"Hello {a}")  
Hello one
```

```
print(f"Hello {a} and {b}")  
Hello one and two
```

```
print(f"Hello {a.upper()} and {b}")  
Hello ONE and two
```

You can use f or F – just be consistent

Use single or double quotes in the parentheses – make sure f or F is outside the quotes

No space between f and the first quote – else gives an error

You put the name of your variable inside { } and the whole function inside (). Use the correct brackets in the correct place

f-string – formatting integers

```
c = int(input("Enter an integer: ")) 70  
d = int(input("Enter another integer: ")) 12
```

```
print(f"Hello {c}")  
Hello 70
```

```
print(f"Hello {c} and {d}")  
Hello 70 and 12
```

```
print(f"Hello {c} and {{d}}")  
Hello 70 and {12}
```

Use single or double quotes in the parentheses

No space between f and the first quote – else gives an error;

Can use F instead of f

Note use of double-braces to output braces

f-string – formatting floats

pi = 3.142579 #setting the variable value without user input

```
print (f"Hello {pi}")  
Hello 3.142579
```

f is inside the parentheses but outside the quotes
No space between f and the first quote – else gives an error
Can use F instead of f
Use single or double quotes in the parentheses

```
print(f"Hello {pi:.2f}")  
Hello 3.14
```

Inside the braces{ } note use of f
pi:.2f means output 3.142579 to 2 decimal places
Note use of colon after var name and use of 'f' after the colon
If we used {pi:.2} we get o/p field of 2 spaces excluding the dpt

f-string – formatting floats

`pi = 3.142579` #setting the variable value without user input

```
print (f"Hello {pi}")  
Hello 3.142579
```

f is inside the parentheses but outside the quotes
No space between f and the first quote – else gives an error
Can use F instead of f
Use single or double quotes in the parentheses

```
print(f"Hello {pi:.2f}")  
Hello 3.14
```

Inside the braces{ } note use of f
pi:.2f means output 3.142579 to 2 decimal places
Note use of colon after var name and use of 'f' after the colon

Computational Thinking

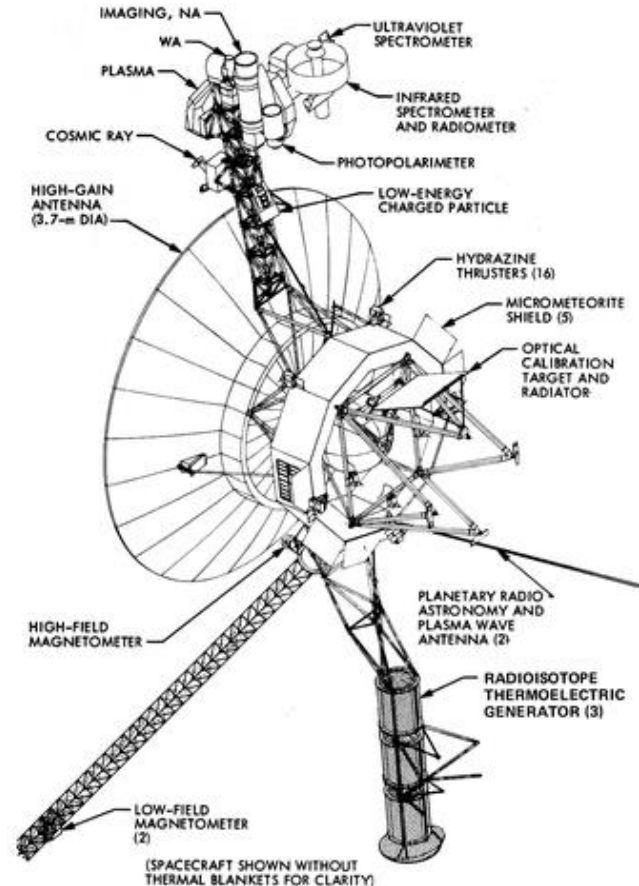
Number of decimal places for precision?

<https://www.jpl.nasa.gov/edu/news/2016/3/16/how-many-decimals-of-pi-do-we-really-need/>

- NASA Jet Propulsion Laboratory
 - Use pi to 15dp for highest accuracy calculations for interplanetary navigation
 - 3.141592653589793 (15 dp)
- pi has mathematical properties with infinite sequence of digits
 - Why don't we use as many dp's as we can (more dp's mean more accuracy?)
 - 15dp is sufficient to understand the cosmos from smallest to largest of scales

Computational Thinking: Voyager Spacecraft: Number of dp's for distance from Earth precision

By NASA/JPL -
"Diagram:
Spacecraft
diagram";
Voyager Bulletin, no.
13, Link, Public
Domain,
<https://commons.wikimedia.org/w/index.php?curid=12618054>



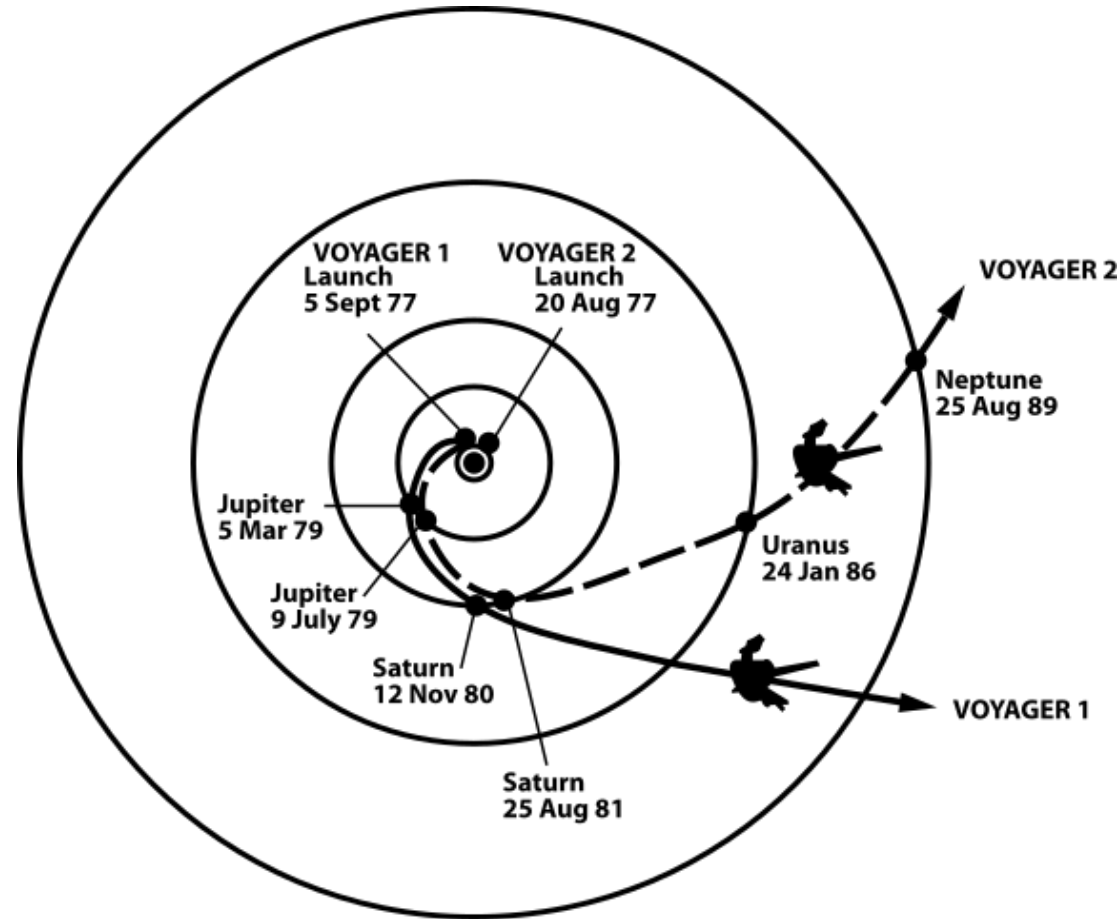
Computational Thinking: Voyager Spacecraft

Number of dp's for precision

- Voyager 1
 - The farthest spacecraft from Earth, launch 1977
 - Now in interstellar space
 - beyond the heliosphere – edge of our solar system
 - Outside of the magnetic effects and particles from our sun
- How accurate can we measure Voyager 1 distance from Earth if we 'only' use 15 decimal places
 - $\pi = 3.141592653589793$
 - ?why don't we use a LOT of dp's – which must increase accuracy?

Computational Thinking: Voyager Spacecraft Trajectory

By Voyager_Path.jpg:
created by
NASA derivative work:
Rehua (talk) - Original
from
http://solarsystem.nasa.gov/multimedia/display.cfm?IM_ID=2143 This
file was derived from:
Voyager Path.jpg;,
Public Domain,
<https://commons.wikimedia.org/w/index.php?curid=18049439>



Computational Thinking

Voyager distance precision

- Voyager 1
 - Earth as the centre of a circle with Voyager 1 at 24 billion km distance from us
 - circle diameter = 48 billion km
 - circumference of the circle = $\pi \times \text{radius} \times 2 = 150$ billion km
- π to 15dp inaccuracy = 1 cm (the width of a little finger)

Computational Thinking: Small vs Large

How many dp's do we need?

- Earth (small)
 - radius = 6,371 km
 - diameter at equator = 12,700 km
 - circumference at equator = 40,100 km
 - pi to 15 dp = 3.141592653589793
 - Inaccuracy = size of a molecule (30,000 times thinner than a human hair)
- Universe (large)
 - radius = 46 billion light years
 - How many digits of pi are needed to calculate the circumference?
 - pi to 37 dp = 3.14159265358979323846264338327950288420
 - Inaccuracy = diameter of a hydrogen atom

Python: Output control

`round()` and `round(num,dp)`

- Python mechanisms to round decimals

```
x= float(input("Enter a float: ")) 5.76543
```

```
round(x)      #answer is 6 - rounds up to nearest integer. When no dp are stated default is zero dp o/p
```

```
round (x,2)   #answer is 5.77 - rounds to 2 decimal places
```

Python: Output control

math.ceil()

- Python mechanisms to round up decimals

import math	#more about this in later slides
math.ceil (1.4)	#answer = 2 (2 is the biggest integer > 1.4)
math.ceil (5.3)	#6
math.ceil (-5.3)	#-5
math.ceil (22.6)	#23
math.ceil (10.0)	10

math.ceil() belongs in the 'math module'.
It rounds a number up to the nearest integer that is greater than the input number.
Answer can't be -6 as this is smaller than the input number

Python: Output control

math.floor()

- Python mechanisms to help round down decimals

import math	#more about this in later slides
math.floor(0.6)	#0 round down to nearest integer
math.floor(1.4)	#1
math.floor(5.3)	#5
math.floor(-5.3)	#-6
math.floor(22.6)	#22
math.floor(10)	#10

math.floor() belongs in the 'math module'.
It rounds a number down to the nearest integer that is less than the input number

STOP

- **You can now stop at this slide**
- **What follows is an explanation only of older styles that we do not use in COMP101**
- **They are here only to explain what you are seeing if you come across older code that uses them**

str.format from v2.6 up to v3.5

For info – do not use

- General syntax

```
print (" Some text{:field width type}" . format (variable) )
```

‘format field’

‘argument’ or

‘argument list’ if several vars

- The format field accepts types

- strings {}
- integer {:d}
- float {:f}

The argument goes in parentheses – it is the name of the var(s) to which user input has been assigned;

The argument is ‘passed’ to the format field;
Remember to use .format - don’t forget the ‘.’
Remember to close the outer parentheses

The format field goes in braces and accepts the argument;
For strings, use {} – no colon and no spaces between { and }
For integer, use {:width d}
For float, use {:width f}

Python v3.5: str.format – formatting string

For info – do not use

name = "John" #fixed variable – no user input used

```
print ("Hello {}".format(name))
```

Hello John

Input is string:

The format field (the braces {}) does not need a colon or 's' in it;

There must be no space in {};

In the print statement, there needs to be a space between "Hello and {} for readable output

Python v3.5: str.format – formatting integer

For info – do not use

```
stud_id = 123
```

```
print ("Student number{:5d}".format(stud_id))
```

Student number 123

Input is integer: `{:5d}` means:
Format the field to accept a signed integer in a field width of 5 places;
Integer 123 takes up three of the five places in the field;
There are two places in front of 123 and after the word 'number'

Pythonv3.5: str.format – integer

For info – do not use

#INPUT

```
num1 = int (input("num1: ") ) 10  
num2 = int (input("num2: " )) 5
```

#CALCULATION

```
result = (num1+num2)
```

#OUTPUT

```
print("Formatted number is{:8d}" .format(result) )  
Formatted number is    15
```

{:8d} means:

Integer field, width = 8 places

Value 15 takes two of those places

There are six spaces between 'is' and 15

- If a number is longer than the field width, the field width is automatically expanded
 - 12 digit number in a field width of 8 still outputs all 12 digits

Python v3.5: str.format – negative integer

For info – do not use

- Negative numbers have a default that outputs the '-' sign
- For -123, in a field width=5, the minus sign takes up a fourth place

```
audit = -123
```

```
print ("Audit total{:5d}".format(audit))
```

```
Audit total -123
```

Integer -123 takes up 4 places (the negative sign takes up one of the places);
It outputs in an integer field width = 5 places
There will be one space in the output before -123 and after the word 'total'

In the print statement, if we put one space after the word 'total' and before the `{:5d}`, we will get two spaces in the output before -123 and after the word 'total'

Python v3.5 str.format - float

For info – do not use

```
price = 10.00
```

```
print ("Price is{:6.2f}".format(price))
```

Price is 10.00

`{:6.2f}` means:

Format the field to accept a float to 2dp in field width = 6 places;

Float 10.00 takes up five of the six places in the field;

The decimal point takes up one of the places;

There is one place in front of 10.00 and after the word 'is'

Python v3.5: str.format – float

For info – do not use

```
num1 = int (input ("num1: ")) 10  
num2 = int (input ("num2: " )) 3
```

```
# true division gives untruncated float  
result = (num1/num2)
```

```
#result is untruncated float = 3 .33333333333333335
```

```
# {:1.2f} means output to 2 dp in a field width of 1
```

```
# field width is too small but auto-formats to minimum places needed to
```

```
#display the output – here 4 places are needed
```

```
print("Formatted number is {:1.2 f }" .format (result) )
```

```
Formatted number is 3.33
```

Two integers are input

They are cast as float by the division

So output field must accept a float (f)

Python v3.5: str.format pass arguments by position

For info – do not use

```
id = int(input("Enter student id: ") ) 1234
```

```
paid = float(input("Enter payment: ") ) 10
```

```
print("Id {:5d} has paid {:5.2f}").format( id, paid ) )
```

Id 1234 has paid 10.00

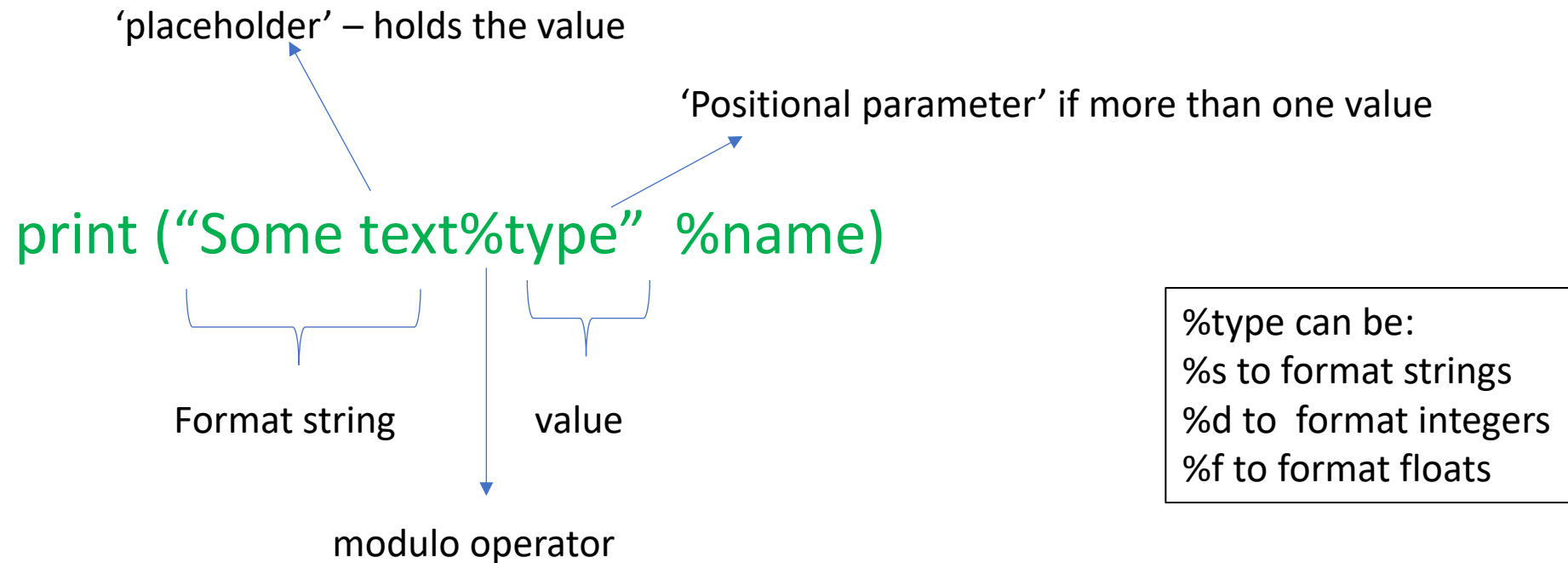
id is first argument in argument list - passed to first format field {:5d}

paid is second argument in argument list – passed to second format field {:5.2f}

Python: % format – v2 to v2.5

For info - do not use

- Old style – quirky and hard to read if many parameters



Python v2.x: % format – do not use string (%f), integer (%d), float (%f)

#string %s

```
name = "John"
print ("Hello %s" %name)
Hello John
```

#integer %d

```
num = 5
print ("Hello %d" %num)
Hello 5
```

This space here puts a space in the output field

#string and int – note ordering

```
name = "John"
num = 5
print ("Hello %s %d" % (name, num))
Hello John 5
```

#float

```
pi_num = 3.14259
print ("Hello %f" % (pi_num))
Hello 3.14259
```

#float in field width of 4 to 2 dp

```
pi_num = 3.14259
print ("Hello %4.2f" % (pi_num))
Hello 3.14
```

Python: str() and repr()

For info – if you find these on the web

- str() gives human-readable output
- repr()
 - Useful for debugging and development
 - Gives unambiguous output
 - Can show possible rounding errors as numbers have better accuracy
 - Gives the 'official' representation of the data as 'seen' by the interpreter
 - Shows string quotes, formatting characters etc

```
print (str(2.0/11.0))  
0.181818181818
```

```
print (repr(2.0/11.0))  
18181818181818182
```